EasyPay — Project Summary
EasyPay is a Python-based hourly pay calculator. The program asks the user for employee name, hours worked, hourly rate, and marital status. It determines regular and overtime hours, calculates regular pay, overtime rate and overtime pay, gross pay, estimated deductions, and net pay. It then displays the results in a structured console summary.

10/11/2026
Project Time: 4.25 Hours

Python Concepts Used:

- user input()
- float()
- variables
- arithmetic operators
- if/else
- comparison operators
- nested conditionals
- round()
- string output with print()
- comments
- string methods including .strip() and .lower()

Debugging:

- Fixed a TypeError caused by attempting to concatenate text and a floating-point number.
- Worked through confusion between string values and numeric values.
- Fixed incorrect or duplicated conditional logic.
- Corrected variable-reassignment problems.
- Debugged syntax errors while building the calculations and decision logic.
- Learned the difference between:
    - = for assignment
    - == for comparison
- Fixed a NameError caused by using comparison syntax where assignment was needed.
- Found and corrected a spelling typo involving "martial" / "marital" during Python practice.
- Worked through problems caused by references to variables that were no longer needed after the program design changed.
- Investigated rounding behavior for overtime calculations and decided to keep the full overtime rate for the calculation before rounding the resulting pay amount.
- Tested calculations repeatedly after changes rather than assuming a change had worked.

Improvements:

- Chose and refined the project concept into an hourly pay calculator.
- Named the project EasyPay.
    - Set up the project in Thonny and saved the working Python file.
    - Added employee-name, hours-worked, hourly-rate, and marital-status inputs.
- Added logic separating:
    - regular hours
    - overtime hours
- Added calculation of:
    - regular pay
    - overtime rate
    - overtime pay
    - gross pay
    - estimated deductions
    - net pay
- Replaced a manually entered deduction percentage with simplified deduction logic based on marital status.
- Established simplified demonstration rates of:
    - 15% for single
    - 12% for married
- Began normalizing marital-status input so capitalization would not affect the result.
- Added comments dividing the program into logical sections.
- Improved console output with blank lines and divider lines so results were easier to read.
- Clearly separated the input, overtime, deduction, calculation, and output portions of the program.
- Ran multiple structured tests, including:
    - under 40 hours
    - exactly 40 hours
    - overtime
    - decimal hours/rates
    - single and married scenarios
- Confirmed all of the core calculation tests passed before moving on to the additional polishing work we did tonight.

Test Scenarios: PASS

Test	Inputs	Expected Results
Under 40 / Single	35 hours, $20.00, Single	Regular: $700.00 OT: $0.00 Gross: $700.00 Deductions: $105.00 Net: $595.00
Exactly 40 / Married	40 hours, $20.00, Married	Regular: $1,000.00 OT: $0.00 Gross: $1,000.00 Deductions: $120.00 Net: $880.00
Overtime / Single	45 hours, $20.00, Single	Regular: $800.00 OT: $150.00 Gross: $950.00 Deductions: $142.50 Net: $807.50
Decimal Hours / Married	42.5 hours, $18.75, Married	Regular: $750.00 OT: $70.31 Gross: $820.31 Deductions: $98.44 Net: $721.87

10/12/2026
Project Time: 3.75 Hours

Python Concepts Used:

- f-strings
- string methods
- input validation
- exception handling
- conditional logic
- comparison operators
- functions / program structure
- Pep 8 concepts

Debugging:

- Fixed an f-string syntax error caused by closing the quotation marks before the formatted variable.
- Fixed a NameError caused by comparing marital status to unquoted words instead of string values.
- Discovered that try/except alone caught bad input but did not re-prompt, leading to the addition of while loops.
- Fixed numeric-validation control flow so invalid values did not accidentally reach break.
- Fixed a missing colon after an elif statement.
- Corrected break placement so the loop exits only after input passes all checks.
- Tested invalid text input such as "seven", "Forty", "Seventeen", "fifty", and other nonsense values.
- Tested invalid numeric boundaries including negative hours/rates, more than 168 hours, rates above $500, and a $0 hourly rate.
- Reconsidered and removed the rejection of 0 hours after recognizing that zero hours can legitimately occur in a pay period.
- Retested the program after moving the entire workflow into main() to ensure the refactor did not change behavior.

Improvements Made:

- Formatted monetary output as proper currency, including:
    - dollar signs
    - thousands separators
    - two decimal places
- Added case-insensitive marital-status input.
- Added whitespace cleanup to marital-status input.
- Added repeated prompting for invalid marital-status entries.
- Added protection against nonnumeric hours-worked input.
- Added protection against nonnumeric hourly-rate input.
- Added realistic range validation:

    - hours worked: 0–168
    - hourly rate: greater than $0 and no more than $500
- Changed wording from “tax rate” to “estimated deduction rate” so the simplified percentages are not presented as actual payroll-tax withholding.
- Improved code readability and PEP 8 compliance.
- Refactored the program into a main() function using the standard Python launcher convention.
- Completed a 7-scenario final test suite, including normal cases, overtime, decimal inputs, boundaries, invalid text, invalid numeric ranges, capitalization/whitespace variations, marital-status rejection, and post-main() regression testing.
- All 7 final scenarios passed.